



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Indian Institute of Technology Jodhpur

CSL7490: Introduction to Blockchain

Instructor: Dr. Susil Kumar Mohanty

Minor Exam

Time: 2:00 hrs.

Date- 19th February, 2026

Maximum Marks: 50

Name and Roll no.:

- All questions are compulsory.
- Clearly and concretely list all assumptions if you are making any.
- A direct answer (result) without any accompanying explanation will receive 0 marks.
- Please aim for precision in your answer to the question. Any unnecessary elaboration or attempt to deceive will result in a 50% deduction in marks.
- If anyone is found copying before, during, or after the exam, he/she will be removed from the course and given a FAILED grade.

Part - A

1. In Bitcoin network, a miner attempts to include a block with a timestamp T_{block} . If the network median time is T_{net} and the Median Time Past of the last 11 blocks is T_{mtp} . For a Bitcoin block to be valid, its timestamp must satisfy the inequality, write the inequality. [1 Mark]

Answer: $T_{mtp} < T_{block} \leq T_{net} + 2 \text{ hours}$

2. Given standard Bitcoin P2PKH script: [1 Mark]

ScriptPubKey: **OP_DUP** OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG

ScriptSig: <Signature> <PublicKey>

Explain why **OP_DUP** is required?

Answer: **OP_DUP** preserves PublicKey for both hash check and signature verification

Without duplication:

- OP_HASH160 would remove PublicKey
- OP_CHECKSIG would have no PublicKey

Thus OP_DUP preserves PublicKey for signature verification.

3. A Bitcoin Script execution ends with stack [-2]. Is it valid? [1 Mark]

Answer: **Yes (any non-zero is TRUE)**

4. An attacker controls 25% hash power and the honest network controls 75%. What is the approximate probability the attacker catches up after 4 confirmations?

Answer: $\left(\frac{25\%}{75\%}\right)^4 = \left(\frac{0.25}{0.75}\right)^4 = (0.333)^4 \approx 0.012 = 1.2\%$

5. If the Bitcoin Network Hash Rate doubles instantly, what specific value in the Block Header will change after the next 2,016-block window to compensate? [1 Mark]

Answer: The **nBits (Target)** value will decrease, making the "Target" smaller and the difficulty higher.

6. Suppose a miner, Miner X, successfully solves the cryptographic puzzle for Block #950,000. Miner X is feeling generous and decides to forgo their reward. They construct a block that contains 2,000

valid user transactions but **does not include a coinbase transaction**. What happens in this situation, give your justification? **[1 Mark]**

Answer: **The block will be rejected by the entire network**. Every full node on the Bitcoin network follows a rule: **The first transaction in every block must be a coinbase transaction**. If this transaction is missing, the block is considered malformed and invalid.

7. A transaction is included in block 850,000. Later, a reorganization replaces blocks 850,000 to 850,003. What is the depth of this reorganization? **[1 Mark]**

Answer: **Blocks replaced:**

850,000
850,001
850,002
850,003

Total: 4 block reorganization

8. Alice wants to set up a 2-of-3 Multi-Sig. She has two choices:
- Raw P2MS: The script is stored in the ScriptPubKey (Output).
 - P2SH: Only the 20-byte hash is stored in the ScriptPubKey.

The P2SH ScriptPubKey is a fixed **23 bytes** (OP_HASH160 + 20-byte hash + OP_EQUAL). A 2-of-3 P2MS ScriptPubKey is a size of **105 bytes** (2 <PubKey1> <PubKey2> <PubKey3> 3 OP_CHECKMULTISIG). By what percentage does P2SH reduce the size of the UTXO compared to raw P2MS for this 2-of-3 setup?

Answer:

$$\text{Reduction} = 1 - \left(\frac{\text{P2MS Size}}{\text{P2SH Size}} \right) = 1 - \left(\frac{23}{105} \right) \approx 1 - 0.219 = 78.1\%$$

9. Suppose a Miner M mines Block 800,000. The coinbase transaction creates two outputs: **2.0 BTC** to Address X and **1.125 BTC** to Address Y. At block height 800,050, Miner M creates a transaction that spends both outputs and sends 1 BTC to an exchange and the rest back as change. Miner M also sets a high fee, and a mining pool tries to include this transaction in Block 800,060. At which earliest block height will the Bitcoin network accept this spending transaction as valid? **[1 Mark]**

Answer: **Coinbase outputs require 100 confirmations before they are spendable**. Coinbase created in block 800,000 becomes spendable starting when the chain height is: $800,000 + 100 = 800,100$. So, the earliest block that can validly include the spending transaction is: **800,100**.

10. When a Bitcoin node validates a new transaction, it must ensure that the inputs are spending existing funds. To do this, the node checks the referenced _____, which is composed of the _____ and the _____. This combination enables an efficient $O(1)$ lookup in the _____ database without scanning the entire blockchain. **[1 Mark]**

Answer: **Outputpoint, TXID (Transaction ID), VOUT (or Output Index), UTXO Set (or Chainstate)**

Part - B

1. A Pruned Node deletes old block data to save space. Does this make it a Light Node? Justify your answer. [2 Mark]

Answer: No. A Pruned Node is still a **Full Node**. It verified every transaction from the Genesis block to the present. Even though it deletes old *block data*, it keeps the entire **UTXO set** (the current state). It can still verify new transactions 100% trustlessly, unlike an SPV node which must trust others for validity.

2. How does **P2SH (Pay-to-Script-Hash)** improve the efficiency of the **UTXO Set** compared to the older **P2MS (Multisig)**? [2 Mark]

Answer: In P2MS, the full list of public keys (which can be large) must be stored in the UTXO set until spent. In P2SH, only a 20-byte hash of the script is stored. This shifts the data burden from the Global Ledger to the individual Spender, keeping the UTXO set lean for all nodes.

3. In a 3-of-5 P2MS transaction, a developer forgets to add the dummy OP_0 required for the OP_CHECKMULTISIG bug.

Stack contains: [Sig1, Sig2, Sig3, 3, Key1, Key2, Key3, Key4, Key5, 5]

When OP_CHECKMULTISIG executes and attempts to pop the *dummy* element after consuming the keys and signatures, which item will it accidentally pop from the stack and why? (Assume the stack has other data below it, like a TX_ID from a previous operation).

[2 Marks]

Answer: It will pop the next item below the signatures. If there is nothing else on the stack, the execution will *fail* with a *Stack Underflow* error, making the transaction *invalid*.

4. The Bitcoin network targets a block every 10 minutes. A difficulty period lasts 2016 blocks. In the last period, the 2016 blocks were mined in exactly 10 days. By what percentage will the difficulty adjust in the next period? [2 Marks]

Answer:

- Expected Time: $2016 \times 10 \text{ minutes} = 20,160 \text{ minutes}$.
- Actual Time: $10 \text{ days} \times 24 \text{ hours} \times 60 \text{ minutes} = 14,400 \text{ minutes}$.
- Adjustment Ratio: $\text{Expected/Actual} = 20,160/14,400 = 1.4$.
- The difficulty will increase by **40%** (multiplied by 1.4).

5. To compute kG for a private key $k = 100$, a miner uses the Binary Add-and-Double algorithm for better efficiency. If a Double operation takes $10\mu\text{s}$ and an Addition takes $15\mu\text{s}$, calculate the total time to compute 100G. [2 Marks]

Answer:

- 100 in binary: $(1100100)_2$.
- Binary steps:
 - 1 (Start)
 - 1: Double + Add
 - 0: Double
 - 0: Double
 - 1: Double + Add
 - 0: Double

- 0: Double
- Counts: **6 Doublings, 2 Additions.**
- Time: $(6 \times 10) + (2 \times 15) = 60 + 30 = \mathbf{90\mu s.}$

Part - C

1. A developer creates a new blockchain called V-Chain. To make it *fast*, they use a hash function with only 80 bits of output. The network has a total hash rate of 2^{40} hashes per second. **[2+3 = 5 Marks]**
 - a. The network's Proof of Work (Preimage Attack) currently requires miners to find a hash with 50 leading zeros. A miner wants to calculate the Expected Time (in seconds) to find a valid block. Calculate this value. **[2 Marks]**
 - b. A malicious *Double-Spend* attacker doesn't want to mine honestly. Instead, they want to use a *Birthday Attack* to generate two different versions of the same block height (*Block_A* and *Block_B*) that result in the exact same 80-bit hash. **[3 Marks]**
 - i. Calculate how many seconds it would take this attacker to have a 50% chance of finding such a collision using the total network hash rate.
 - ii. Based on your answer, explain why Bitcoin uses SHA-256 (256-bit) for its PoW even though the current difficulty only requires finding ~70-80 zeros.

Answer:

- a. Preimage Calculation:
 - The difficulty is 2^{50} attempts. The hash rate is 2^{40} hashes/sec.
 - $\text{Time} = \frac{\text{Difficulty}}{\text{Hash Rate}}$
 - $\text{Time} = \frac{2^{50}}{2^{40}} = 2^{10} = \mathbf{1024 \text{ seconds } (\approx 17 \text{ minutes})}$
 - b. Birthday Attack & Security Analysis:
 - To find a collision in an 80-bit hash ($n = 80$), the Birthday Paradox says we need $\sqrt{2^{80}} = 2^{40}$ attempts.
 - Time to Collision: $\frac{\text{Attempts Needed}}{\text{Hash Rate}} = \frac{2^{40}}{2^{40}} = \mathbf{1 \text{ second.}}$
 - **The Problem:** While it takes the honest network 1,024 seconds to find a block (Preimage), an attacker can find a collision in just 1 second (Birthday).
 - **The Bitcoin Lesson:** If Bitcoin used an 80-bit hash, even if the mining difficulty was high, an attacker could easily create two different transaction sets that hash to the same value. They could show *Block A* to a merchant and later broadcast *Block B* (which sends the money back to the attacker) to the network. Both would be *valid* in the Merkle Tree.
 - **We use 256-bit hashes so that the Birthday bound 2^{128} remains impossibly high, ensuring that a collision is never found by accident or by a malicious actor within the lifetime of the universe.**
2. A Bitcoin block contains exactly $N = 1024$ transactions. An SPV node needs to verify that Transaction #452 is included in this block. Given that, each hash in the Merkle Tree is used SHA256 hash function. The Full Block size is 1 MB. The Block Header is 80 bytes. **[1+2+2 = 5 Marks]**
 - a. Calculate the height of the Merkle Tree (the number of levels above the leaves). **[1 Mark]**
 - b. Calculate the total number of bytes the SPV node must download to verify this one transaction (including the header and the Merkle branch). **[2 Marks]**
 - c. Calculate the Efficiency Ratio: What percentage of the full block data does the SPV node need to download? **[2 Marks]**

Answer:

a. Tree Height:

In a binary Merkle Tree, the height **h** is calculated as: **$h = \log_2(N)$**

$h = \log_2(N) = \log_2(1024) = \log_2(2^{10}) = \mathbf{10 \text{ levels}}$ (There are 10 hashes in the path from the transaction to the root)

b. Total Data Downloaded:

To verify the transaction, the SPV node needs:

- The Block Header: 80 bytes.
- The Merkle Branch: 10 hashes $\times$ 32 bytes/hash = 320 bytes.
- **Total: 80 + 320 = 400 bytes.**

c. Efficiency Ratio:

$$\begin{aligned} \text{Ratio} &= \left(\frac{\text{SPV Data}}{\text{Full Block Size}} \right) \times 100 \\ \Rightarrow \text{Ratio} &= \left(\frac{400 \text{ bytes}}{1,000,000 \text{ bytes}} \right) \times 100 \\ &\Rightarrow \mathbf{\text{Ratio} = 0.04\%} \end{aligned}$$

The SPV node only needs **0.04%** of the block's data to achieve cryptographic certainty of inclusion.

Part - D

1. In Bitcoin, ECDSA is used to sign a transaction hash $\mathbf{z}$ (message digest). Let the private key be $\mathbf{d} \in \{1, \dots, n-1\}$, the public key be $\mathbf{Q} = \mathbf{d} \cdot \mathbf{G}$, and the signature be $(\mathbf{r}, \mathbf{s})$ over a group of order $\mathbf{n}$. A miner mistakenly reuses the same nonce $\mathbf{k}$ while signing two different transaction hashes $\mathbf{z}_1$ and $\mathbf{z}_2$. The attacker observes two signatures:

- Signature 1: $(\mathbf{r}, \mathbf{s}_1)$ on transaction hash $\mathbf{z}_1$
- Signature 2: $(\mathbf{r}, \mathbf{s}_2)$ on transaction hash $\mathbf{z}_2$

and notices that the $\mathbf{r}$ values are identical (because the same $\mathbf{k}$ was used).

- Show the mathematical formula the attacker would use to recover the secret nonce $\mathbf{k}$ using $(\mathbf{r}, \mathbf{s}_1)$ and $(\mathbf{r}, \mathbf{s}_2)$.
- Show the mathematical formula the attacker would use to recover the private key $\mathbf{d}$.
- Solve for $\mathbf{k}$ and $\mathbf{d}$ for the following simplified values (all arithmetic $\text{mod } \mathbf{n}$):

- $\mathbf{n} = 13$
- Transaction 1: $\mathbf{z}_1 = 5, \mathbf{s}_1 = 7$
- Transaction 2: $\mathbf{z}_2 = 2, \mathbf{s}_2 = 1$
- Shared value: $\mathbf{r} = 8$ (same for both signatures).

[1+2+1+3+3 = 10 Marks]

Answer:

- Recovering the nonce $\mathbf{k}$:

From ECDSA Signature:

[1 Mark]

- Signature 1: $\mathbf{s}_1 \equiv k^{-1}(\mathbf{z}_1 + \mathbf{r} \cdot \mathbf{d}) \pmod{\mathbf{n}}$
- Signature 2: $\mathbf{s}_2 \equiv k^{-1}(\mathbf{z}_2 + \mathbf{r} \cdot \mathbf{d}) \pmod{\mathbf{n}}$

Multiply both sides by $\mathbf{k}$:

- $\mathbf{k}\mathbf{s}_1 \equiv (\mathbf{z}_1 + \mathbf{r} \cdot \mathbf{d}) \pmod{\mathbf{n}}$ ----- (eqn 1)
- $\mathbf{k}\mathbf{s}_2 \equiv (\mathbf{z}_2 + \mathbf{r} \cdot \mathbf{d}) \pmod{\mathbf{n}}$ ----- (eqn 2)

Subtract the second from the first (the $\mathbf{r}\mathbf{d}$ cancels):

$$\begin{aligned} \mathbf{k}\mathbf{s}_1 - \mathbf{k}\mathbf{s}_2 &\equiv (\mathbf{z}_1 - \mathbf{z}_2) \pmod{\mathbf{n}} \\ \Rightarrow \mathbf{k}(\mathbf{s}_1 - \mathbf{s}_2) &\equiv (\mathbf{z}_1 - \mathbf{z}_2) \pmod{\mathbf{n}} \\ \Rightarrow \mathbf{k} &\equiv (\mathbf{s}_1 - \mathbf{s}_2)^{-1}(\mathbf{z}_1 - \mathbf{z}_2) \pmod{\mathbf{n}} \end{aligned}$$

[2 Marks]

- Recovering the private key $\mathbf{d}$:

From eqn 1 or eqn 2,

$$\mathbf{k}\mathbf{s}_1 \equiv (\mathbf{z}_1 + \mathbf{r} \cdot \mathbf{d}) \pmod{\mathbf{n}}$$

$$\Rightarrow r \cdot d \equiv (ks_1 - z_1) \pmod{n}$$

$$\Rightarrow d \equiv r^{-1}(ks_1 - z_1) \pmod{n}$$

[1 Mark]

c. Given

- $n = 13$
- $z_1 = 5, s_1 = 7$
- $z_2 = 2, s_2 = 1$
 - $r = 8$

i. Compute k :

$$k \equiv (s_1 - s_2)^{-1}(z_1 - z_2) \pmod{n}$$

$$\Rightarrow k \equiv (7 - 1)^{-1}(5 - 2) \pmod{13}$$

$$\Rightarrow k \equiv 6^{-1} \cdot 3 \pmod{13}$$

Check: $6 \times 11 = 66 \equiv 1 \pmod{13}$ (since 65 is divisible by 13).
[Using Extended Euclidean Algorithm]

So, $6^{-1} \equiv 11 \pmod{13}$

Now, $k \equiv 11 \cdot 3 \pmod{13}$

$$k \equiv 33 \pmod{13}$$

$$k \equiv 7 \pmod{13}$$

[3 Marks]

ii. Compute d :

$$d \equiv r^{-1}(ks_1 - z_1) \pmod{n}$$

$$\Rightarrow d \equiv 8^{-1}(7 \cdot 7 - 5) \pmod{13}$$

$$\Rightarrow d \equiv 8^{-1} \cdot 44 \pmod{13}$$

Check: $8 \times 5 = 40 \equiv 1 \pmod{13}$ (since 39 is divisible by 13).
[Using Extended Euclidean Algorithm]

So, $8^{-1} \equiv 5 \pmod{13}$

Now, $d \equiv 5 \cdot 44 \pmod{13}$

$$\Rightarrow d \equiv 220 \pmod{13}$$

$$\Rightarrow d \equiv 12 \pmod{13}$$

[3 Marks]

2. A developer is working with P2SH transactions in Bitcoin and wants to analyze stack behavior during execution. [4+6 = 10 Marks]

a. Stack Execution and Maximum Height [4 Marks]

Consider a P2SH output whose Redeem Script is:

- ScriptSig: **2 3 <RedeemScript>**
- ScriptPubKey: **OP_HASH160 <Hash160 (RedeemScript)> OP_EQUAL**
- Redeem Script: **OP_ADD 5 OP_EQUAL**

Show the complete step-by-step stack trace for:

- ScriptSig execution
- ScriptPubKey execution
- Redeem Script execution

Clearly identify the **maximum stack height** reached during the entire execution process.

b. Stack Timing and Logical Analysis in P2SH [6 Marks]

A student attempts to modify a P2SH locking script as follows:

- Assume that after ScriptSig execution, the stack: **[X, <RedeemScript>]**
- ScriptPubKey: **OP_HASH160 <Hash160 (RedeemScript)> OP_EQUAL OP_DUP**
- Redeem Script: **OP_IF**

...
OP_ENDIF

The student argues that since **OP_DUP** appears in the ScriptPubKey, a duplicated true value will remain on the stack and can be used by the Redeem Script's

OP_IF as a conditional input.

- Does this logic: work correctly, fail completely, or behave unexpectedly? Justify your answer clearly by explaining:
- The execution order of Redeem Script, ScriptSig, and ScriptPubKey in P2SH.
- When the Redeem Script is executed in relation to ScriptPubKey.
- What items are consumed and what remains on the stack during the execution.
- Whether values generated inside ScriptPubKey can be safely relied upon inside the Redeem Script, and why.

Answer: A

Phase 1.a: Execute ScriptSig

- Start:** [] (height 0)
- Push 2: [2] (height 1)
- Push 3: [2, 3] (height 2)
- Push <RedeemScript> (as data): [2, 3, <RedeemScript>] (height 3)

Step	Instruction	Stack (Bottom → Top)	Height
0	Start	[]	0
1	Push 2	[2]	1
2	Push 3	[2, 3]	2
3	Push <RedeemScript>	[2, 3, <RedeemScript>]	3

Phase 1.b: Execute ScriptPubKey (P2SH hash check)

ScriptPubKey: OP_HASH160 <Hash> OP_EQUAL

(where <Hash> = Hash160(RedeemScript) stored in the output)

Current stack before ScriptPubKey: [2, 3, <RedeemScript>]

- OP_HASH160:**
 - Pops: <RedeemScript>
 - Pushes: **H** = **HASH160**(<RedeemScript>)
 - Stack becomes: [2, 3, **H**] (height 3)
- Push <Hash> (constant from ScriptPubKey): [2, 3, **H**, <Hash>] (height 4)
This is the maximum stack height during the entire execution.
- OP_EQUAL:**
 - Pops: <Hash> and **H**
 - Pushes: **True** (assuming they match)
 - Stack: [2, 3, **True**] (height 3)

ScriptPubKey succeeds (top is **True**).

The Implicit Copy Transition:

- The protocol sees **True** and the P2SH pattern.
- It copies the stack as it was after the Phase 1.a **ScriptSig** but before **Phase 2 started**.
- Phase 2 Stack (Initial): [2, 3] (The RedeemScript is popped off to be executed).

Step	Instruction	Stack (Bottom → Top)	Height
4	OP_HASH160	[2, 3, H]	3
5	Push <Hash>	[2, 3, H, <Hash>]	4
6	OP_EQUAL	[2, 3, True]	3

- **Maximum Stack Height = 4**
- Occurs at **Step 5** during ScriptPubKey execution.
- Final result = [True] → Transaction Valid.

Phase 2: Execute Redeem Script (P2SH rule)

After ScriptPubKey passes, the interpreter executes the Redeem Script using the stack state saved right after ScriptSig, with <RedeemScript> removed. So the stack for Redeem Script execution is restored to: [2, 3]

Now run Redeem Script: OP_ADD 5 OP_EQUAL

7. OP_ADD
 - Pops: 3 and 2
 - Pushes: 5
 - Stack: [5] (height 1)
8. Push 5: [5, 5] (height 2)
9. OP_EQUAL
 - Pops: 5 and 5
 - Pushes: True
 - Stack: [True] (height 1)

Redeem Script succeeds.

Step	Instruction	Stack (Bottom → Top)	Height
7	Restore stack	[2, 3]	2
8	OP_ADD	[5]	1
9	Push 5	[5, 5]	2
10	OP_EQUAL	[True]	1

Answer: B

1. Execution Order in P2SH

In a P2SH spend, execution happens in this order:

1. **ScriptSig** executes
2. **ScriptPubKey** executes
3. If ScriptPubKey evaluates to TRUE and is P2SH, then the **Redeem Script** is executed.

This order is extremely important.

2. What Happens During ScriptPubKey

ScriptPubKey: OP_HASH160 <Hash> OP_EQUAL OP_DUP

Assume after ScriptSig execution: [X, <RedeemScript>]

Execute ScriptPubKey:

- OP_HASH160
 - Pops <RedeemScript>
 - Pushes H
 - Stack: [X, H]

Push <Hash>

- Stack: [X, H, <Hash>]

OP_EQUAL

- Compares H and <Hash>
- Pushes True (if match)
- Stack: [X, True]

OP_DUP

- Stack: [X, True, True]

a. Does the logic work, fail completely, or behave unexpectedly? [1 Mark]

- **Fail completely** (as a P2SH design).
- Because the locking script is no longer P2SH once you append **OP_DUP**.
- Reason: Bitcoin treats an output as P2SH only if ScriptPubKey is exactly:
- **OP_HASH160 <20-byte Hash160> OP_EQUAL**
- In Question: **OP_HASH160 < Hash160 (RedeemScript)> OP_EQUAL OP_DUP**

Now it does not the **P2SH** template, therefore Bitcoin will not execute the Redeem Script at all.

b. Execution order of Redeem Script, ScriptSig, ScriptPubKey in P2SH [1 Mark]

In general, P2SH order (only when ScriptPubKey matches P2SH template):

- ScriptSig
- ScriptPubKey
- Redeem Script (only if ScriptPubKey is exact P2SH form and succeeds)

In the modified script case:

- ScriptSig
- ScriptPubKey
- Stops here (Redeem Script is not executed)

c. When is the Redeem Script executed in relation to ScriptPubKey? [1 Mark]

In general, P2SH: Redeem Script executes after ScriptPubKey succeeds and ScriptPubKey matches the exact P2SH template.

In the modified script case: ScriptPubKey has extra **OP_DUP**, so Redeem Script is never executed.

d. What items are consumed and what remains on the stack? [2 Mark]

- After ScriptSig, stack is: [X, <RedeemScript>]
- After ScriptPubKey, stack is: [X, True, True]

But Redeem Script is still not executed.

e. Can ScriptPubKey-generated values be safely relied upon inside Redeem Script? [1 Mark]

No. Not valid here, because Redeem Script does not run at all.

There is nothing to rely on, because redeem script execution never happens when the ScriptPubKey is **not recognized as P2SH**.